Group Final Project: Agent 系统设计与实现

1. 项目意义

基于 LLM 的 Agent 利用 LLM 推理能力，利用工具，长短记忆以及设计模式，实现具有自主能力的智能体。真实业务通常按照一定流程进行，对多智能体进行编排形成工作流已成为 Agent 落地最普及形式。以业务出发，设计和实现复杂智能体对技术挑战性要求有较大差异。编程 Agent 已成为典型落地场景，对 LLM 模型本身要求极高，另外对系统架构也提出新要求。实践表明，以上下文工程（context engineering）为代表的 Agent Engineering 技术话题日益精深。为执行编程以及机器人具身智能的长程任务，基于已有 LLM 模型必须设计新的系统架构（最新进展为 DeepAgent 以及 Skills）。

面向本科生基础能力训练，从源代码级别理解多 agent 编排和执行过程对夯实基础大有裨益。Final Project 从 Agent Engineering 角度，涵盖 Agent 系统设计（多 Agent 编排）、长短期记忆（Workflow 中 Agent 上下文及数据存储），人在环路（Human-in-the-Loop）交互以及系统行为观察和评估等。

以 OpenAI Agent 开源示范应用（customer service）出发，通过完善 Demo 系统，学习和实践 Agent 系统构建涉及的诸多工程问题，提升同学工科完整思维和实践能力。此外，为开放话题留出空间，欢迎同学以 OpenAI Agent SDK 为开发工具，改造开源或自己实现其它多 Agent 系统，要求涵盖 Agent 系统设计（多 Agent 编排）、长短期记忆（Workflow 中 Agent 上下文及数据存储），人在环路（Human-in-the-Loop）交互以及系统行为观察和评估等技术要点。

2. 技术要点

技术要点 1： 短期和长期存储 【参见代码注释】

Agent 共享的上下文数据结构进行永久存储，构成系统的 Agent 涉及的函数、工具、mcp 集成等相关操作，通过真实数据或者仿真数据执行（建立存储数据结构）。系统功能更逼近真实业务场景，提升系统可用性。

技术要点 2：人在环路交互及用户反馈 【参见代码注释】

用户提供过前端界面对 AI 响应消息进行评分反馈，即对 Agent 回复消息进行喜好评价，并将 trace id 对应的响应消息的评分更新至 langfuse 平台。

基本思路：将 langfuse 的 trace id 绑定 agent 消息反馈，实现后端和前端共享 trace id 及用户反馈状态，调用 langfuse 打分函数将用户反馈的评分上传至 langfuse 平台。

实现策略：（a）前端和后端均运行 langfuse，后端生成 trace id 和其他信息，前端根据 trace id 进行评分；（b）后端运行 langfuse，通过 trace id 实现打分消息绑定通信，fastapi 后端更新打分结果至 langfuse 平台。

技术要点 3：基于观测平台进行评估【Agent 评估流程和 langfuse 技术文档】

（a）专家利用观测平台提供的 annotation 工具修改 trace 评分（学生可自行模拟专家完成实验）。
（b）LLM-as-a-Judge 提供的 evaluator（langchain 等平台还可提供 agent 的 函数/工具调用轨迹 trajectories 的评估），对 agent 响应消息进行评分。
（c）Score 分析。通过平台提供的 score 分析工具，观察评估指标提升（人工或者实验打分），理解 observation 和 evaluation 在智能体构建中逐步优化提升的重要作用。

技术要点 4：业务优化及扩展【参考给定源代码及 customer service demo 最新代码】

优化已有 agent 并增加新的 agent，提升系统业务处理能力。对于给定的航班客服系统（customer service），建议增加 food agent 来处理用户饮食需求。

技术要点 5：界面外观提升【前端编程】

提升前端界面的美观度，系统运行效率等。

3. 基础系统选择

3.1 课程提供的 customer service agent 工具包

完善 Agent 系统，工作涵盖第二部分涵盖的 4 个要点。

3.2 其它基于 OpenAI Agent SDK 的 Agent 开源系统

DeepResearch Agent, Coding Agent 等最新类型 Agent 系统， 它们的 Agent 编排架构，Agent 系统能力，代码难度等不低于给定的 customer service 应用。小组选定后与任课教师联系。对原型系统起点难度高，工作涵盖所有技术要点，最终系统完成度好的小组，可有 10%-30%的加分。

4. 评分方法

4.1 总体评分（20%）

系统外观，运行时以及系统能力。

4.2 技术要点打分 （60%）

通过文档阅读，运行时表现及源代码查看相结合。

4.3 文档打分（20%）

提交报告格式不限，包括：系统简介（含 Agent Graph），技术要点说明（四个技术要点的设计及实现，langfuse 相关部分须有截图），自评亮点（小组工作突出之处）。

4.4 加分项（10-30%）

任课教师和小组充分技术讨论后确定。

5. 工作任务表

每个小组提供人员和任务分配表。

附录 A：

OpenAI Customer Service Agents 安装说明（含 python 前端）

6. 项目简介

OpenAI Agent SDK 是一款轻量级，开源且功能强大的 Agent SDK, 配合 OpenAI 开发者平台的 Agent Builder 工具，可构建具有全部源代码的 Agent 工作流，超越 Agent Builder(Langchain)/GoogleADK/ Coze/Dify/n8n 等无法查看全部源代码的 Agent 可视编排框架。OpenAI Customer Service Agents Demo 示范企业级客服 Agent 系统设计与实现，是 OpenAI Agent SDK 若干开源项目中的明星项目(https://github.com/openai/openai-cs-agents-demo)

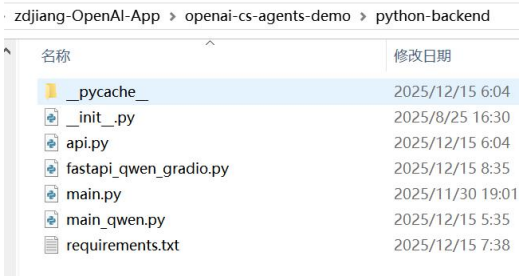
7. 代码特点

基于该开源项目最新 Chatkit 之前的 2025 年 8 月版本，该版本代码未采用 openai 最新的 chatkit 重构，代码结构清晰 简洁易懂，充分展示 Agent 编排实现的工作流原理和设计。

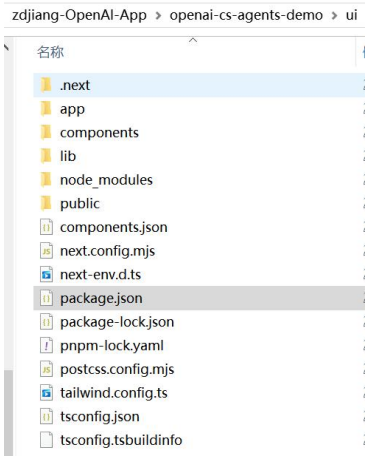
教学团队通过适配 Qwen 模型，开发基于 Gradio UI 的 python 前端，方便同学进行代码阅读和实时运行时探究。Fastapi 服务端口及 Agent 定义等关键代码部分增加注释及作业要求，方便代码阅读，实现编程任务。

8. 编译运行

1. 解压 openai-cs-agents-demo.zip
2. 依赖包安装 1：Fastapi 后端安装需求见 requirements.txt，位于 python-backend 目录下：

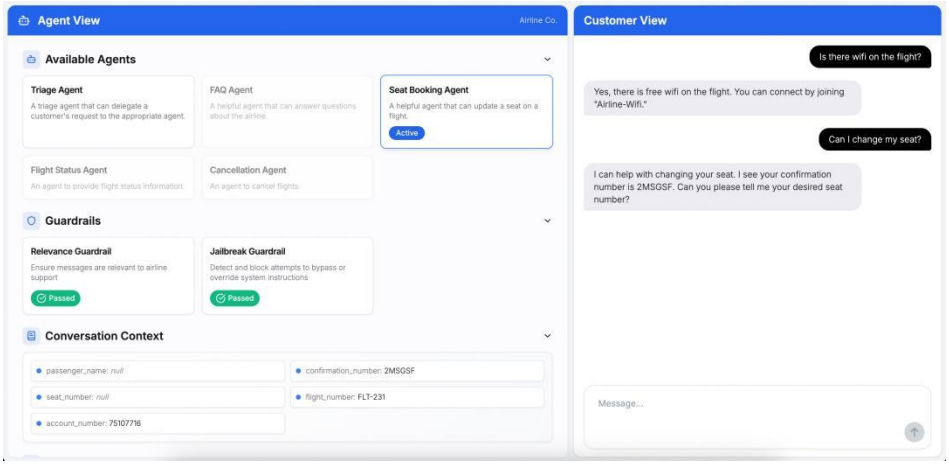
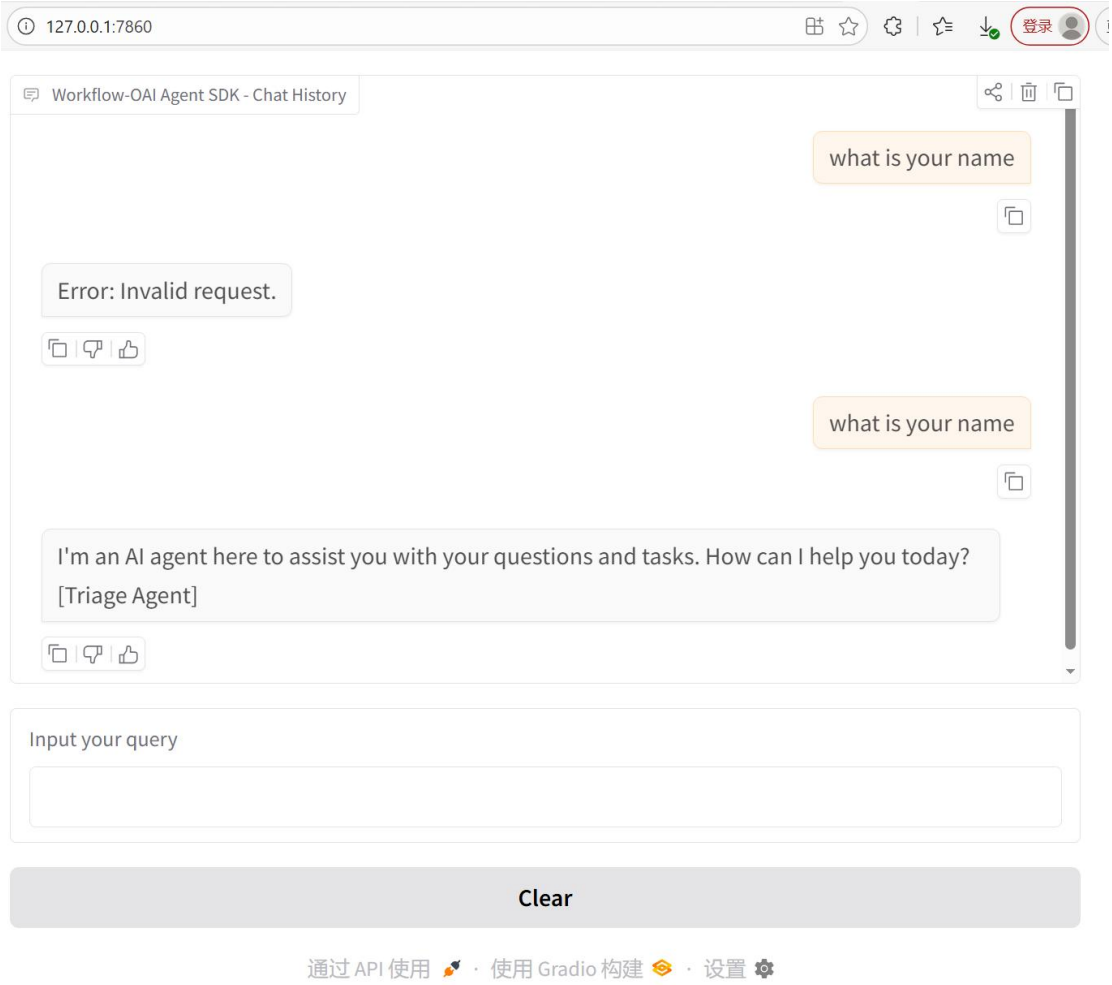


3. 依赖包安装 2：Next.js 前端安装需求见 package.json，位于 ui 目录下，该所需 next.js 相关版本虽不新，可正确运行。



4. Fastapi 服务端：Fastapi endpoint 服务（8000 端口），VS Code 打开项目目录， 设置虚拟环境后，进入 python-backend 目录，执行： python -m uvicorn api:app --reload --port 8000

5. Next.js 前端：VS Code 虚拟环境下，进入 ui 目录下，执行: npx next dev



6. 基于 gradio UI 的 python 前端：VS Code 虚拟环境下，进入 python-backend 目录，执行：python .\fastapi_qwen_gradio.py，运行画面见如下截图（显示具有一定容错能力，如服务器未启动显示 Invalid request）